INSPECTAI / SYSTEM ARCHITECTURE REPORT

How InspectAI Is Built

A full-stack architecture reference for the NL2SQL platform: the two-model agent pipeline, the domain-split backend, the validation and RBAC layers, and the Angular client that talks to it.

Scope: `khidmah-nl2sql-prototype/` — backend (`backend/src`) and frontend (`frontend/src/app`)

Compiled: 2026-08-05, via direct source verification of both trees (not documentation alone)

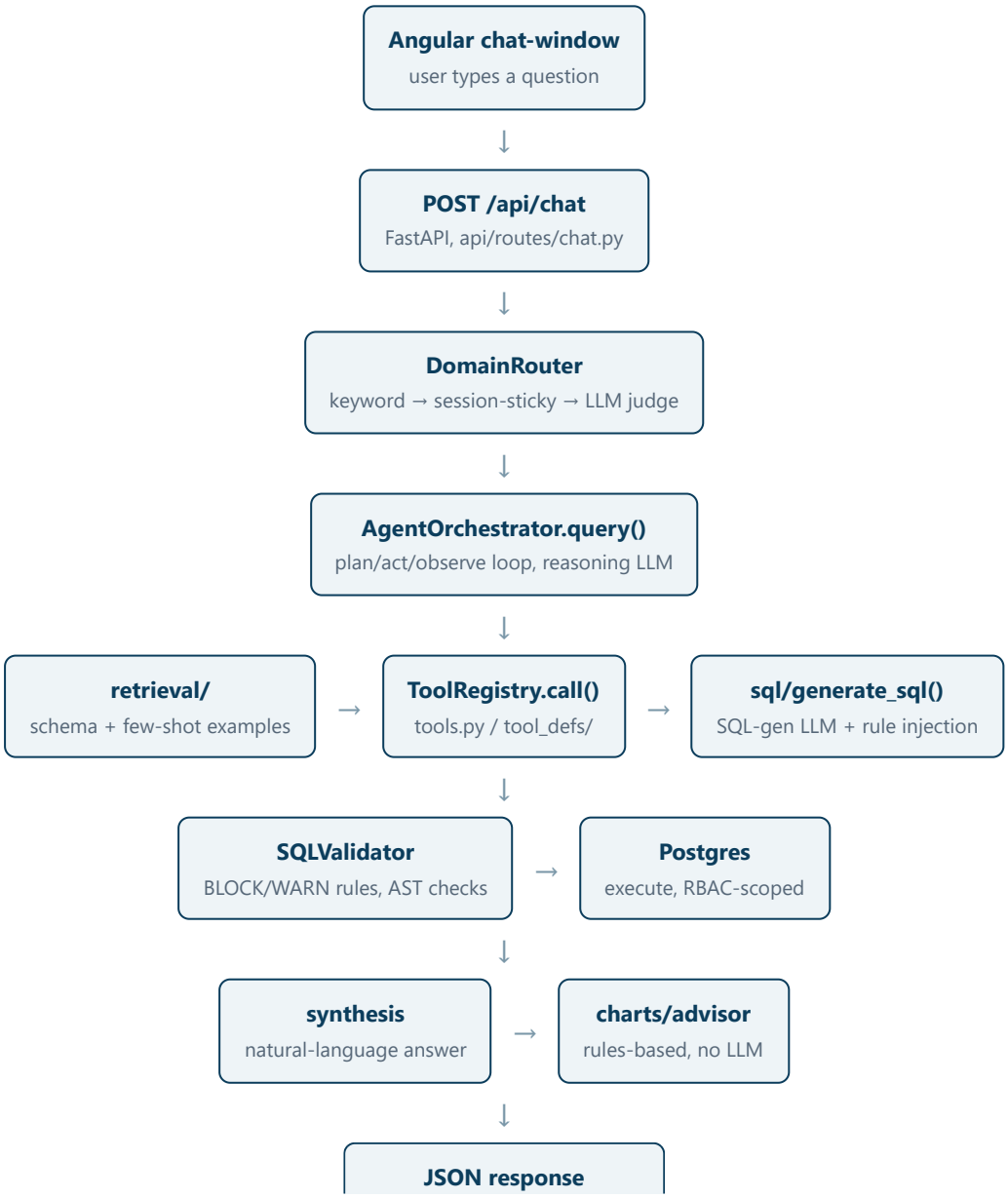
Purpose: reference material for engineers onboarding to the codebase

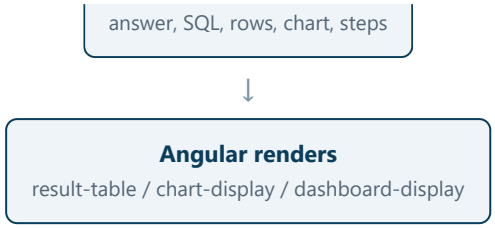
1 What This System Is

InspectAI is a natural-language-to-SQL (NL2SQL) platform for a facility inspection and audit workflow. A user types a question in a chat interface ("which facility has the most open corrective actions?"); the backend plans the request, generates SQL against a live Postgres schema, validates and executes it, and returns a natural-language answer alongside the raw data, the SQL used, and an optional chart. The system covers two related but structurally separate business domains — **Inspection** and **Audit** — routed automatically per question.

2	2	~90	Angular 21
BUSINESS DOMAINS (INSPECTION / AUDIT)	LLMS IN THE LOOP (REASONING + SQL-GEN)	BACKEND PYTHON FILES ACROSS 9 PACKAGES	FRONTEND, STANDALONE COMPONENTS

2 End-to-End Request Flow





This mirrors the package-layout diagram in the project's own README, which is accurate at the package-boundary level; the sections below go one level deeper into each stage and flag a handful of places where the documentation has drifted from what the code actually does.

3 Backend Architecture

3.1 The Two-LLM Split: Orchestrator + Generator

The system deliberately uses two different LLMs for two different jobs, rather than one model doing everything: a **reasoning LLM** that plans which tool to call next (never writes SQL itself), and a **SQL-generation LLM**, specialized for turning a schema hint into a query. This split exists because a general-purpose reasoning model and a code/SQL-specialist model have different strengths, and mixing the two responsibilities in one prompt was the original source of several early accuracy problems.

> **DOCUMENTATION DRIFT** The project README names the orchestrator/generator pair as "Llama 3.1 8B" and "DeepSeek Coder V2". That is stale. `config/settings.py` 's hardcoded defaults are `qwen2.5:14b-instruct-q4_K_M` / `sqlcoder:7b`; the actual configured deployment (confirmed both from code comments and directly, via live runs against this environment) runs `qwen3.5:9b` for reasoning and `qwen2.5-coder` for SQL generation. A comment in `sql/generation.py` notes the "deepseek" naming is a legacy label surviving in code from before the project moved off DeepSeek. The system also fully supports an Anthropic mode (`llm_provider="anthropic"`), where a single Claude model (default `claude-sonnet-5`) serves both roles, since Claude has no small SQL-specialist counterpart to pair with a larger reasoner the way the Ollama setup does.

`agent/orchestrator/loop.py` 's `AgentOrchestrator.query()` is a plan/act/observe loop capped at 8 iterations. Each iteration: call the reasoning LLM → parse its tool-call JSON → run a handful of deterministic auto-redirect guards (disambiguation/ordinal backstops, an observations-vs-findings redirect) → dispatch to `ToolRegistry` → feed the result back as the next turn unless the tool is **terminal**. `generate_sql` is special-cased: on success it auto-executes the SQL immediately (skipping a separate "now call execute_sql" round trip); after three validation failures it force-executes anyway and lets Postgres be the final arbiter rather than looping forever. If all 8 iterations are exhausted, a forced-summary prompt produces a fallback answer instead of failing silently.

3.2 Domain System: Inspection vs. Audit

`agent/domains.py`'s `DomainConfig` bundles everything that differs between the two domains: system prompt, SQL-generation prompt, seed few-shot examples, rules engine, schema introspection function, join-graph module, and RBAC scope table. Two fully separate content modules back this (e.g. `agent/prompts/builder.py` vs. `builder_audit.py`).

Routing — `agent/domain_router.py`

Tier	Mechanism	When it fires
1	Regex keyword match	Instant, no LLM call — matches domain vocabulary ("audit"/"auditor"/"corrective action" vs. "inspect"/"mitigative action"), with a backstop for ambiguous synonyms like "remedial/curative actions".
2	Session stickiness	No keyword match — falls back to whichever domain the conversation was already in.
3	LLM judge	No keyword and no prior conversation — one short JSON completion on the already-warm reasoning LLM; returns an explicit `AMBIGUOUS` + clarifying question when genuinely unsure, rather than guessing.

Per `api/app.py`'s startup lifespan, the two domain orchestrator instances **share** one DB connection pool, one embedding model, and one warm reasoning-LLM client — but each builds its own `SQLValidator`, `RulesEngine`, seed-example index, prompts, join graph, and SQL-LLM client instance.

3.3 Tool Framework

Two layers coexist. A shared declaration framework (`agent/tool_defs/` — `ToolDef/ArgSpec` frozen dataclasses: name, args, doc, terminal flag, domains, optional context provider) currently backs two purpose-built tools. Eight longer-standing tools are hand-dispatched directly in `ToolRegistry` (`agent/tools.py`).

Tool	Purpose
`resolve_ordinal_reference`	Resolves "the first one" / "the 3rd item" to a real row id — LLM supplies a clean `position` instead of the code guessing from text.
`resolve_disambiguation_reply`	Resolves a user's pick ("the second George") from a prior name-collision list.
`list_forms`	Distinct form/module names available, domain-scoped.
`semantic_search`	Embedding search over indexed question labels.
`search_questions`	Keyword (ILIKE) search over question labels.
`get_answers`	Paginated query over the flattened answers table(s), RBAC-scope-aware.
`get_answer_stats`	Numeric/categorical aggregate stats over answers.
`generate_sql`	The core NL2SQL step — see §3.4.
`execute_sql`	Runs generated SQL against Postgres.
`get_schema`	Curated or raw column introspection for a named table.

`agent/tool_dispatcher.py` 's `parse_tool_call()` does multi-stage recovery of the reasoning LLM's tool-call JSON (fence stripping, repair library, brace-scanning, regex fallback), including two targeted repairs for known small-model corruption patterns (fragmented schema-hint and fragmented final-answer text from the model breaking a long string across multiple JSON tokens).

3.4 SQL Pipeline

`sql/generation.py`'s `generate_sql()` assembles a schema hint (curated slim block, or full per-table detail when specific tables are already known), pulls retrieved few-shot examples, fills the prompt template, and calls the SQL-generation LLM. Three deterministic guards can force one corrective retry after generation: an offset guard (preserves LIMIT/OFFSET when resolving an ordinal), a **table guard** (enforces a fired rule's required/forbidden tables — the exact mechanism reworked today, see the companion session report), and a UUID-filter guard (pins an already-resolved identity filter so a retry can't silently drop it). After generation: registry autocorrect rules, an AST-based structural repair pass (sqlglot), several targeted autocorrects, then validation.

Validation — `sql/validation/`

`SQLValidator` composes six mixins (cleaner, identity, intent-autocorrect, shape, semantic, scope) and runs registry rules plus roughly ten hand-written checks (hallucinated columns, impossible joins, UUID-in-SELECT, display-code/UUID confusion, undefined aliases, ungrouped-aggregate columns, stale date literals), mostly via sqlglot AST inspection with regex fallback. Every issue carries a severity: `HARD_FAIL` (blocks execution, feeds a retry prompt) or `SOFT_WARN` (logged only, never shown to the LLM) — a deliberate design choice so heuristic-but-not-certain checks can't reject correct SQL.

Business rules — `sql/rules/`

A `RulesEngine` loads four rule types per domain — `BLOCK`, `WARN`, `AUTOCORRECT`, `INJECT` — from two fully separate registries (inspection: ~32 INJECT rules across several files; audit: `audit_rules.py`, 8 INJECT rules). INJECT rules match business vocabulary in the question and inject schema-specific SQL guidance (budget-capped at 4,500 characters) plus, optionally, tables the resulting SQL *must* or *must not* contain — enforced post-generation by the table guard mentioned above. `SQLValidator` only ever activates registry rules through an explicit allowlist, so a newly-added rule never silently starts firing without deliberate opt-in.

Seed examples — `sql/examples/`

Two hand-curated NL→SQL corpora (inspection, audit) feed the retrieval-reranking index described next.

3.5 Retrieval

`retrieval/questions.py`'s `SemanticQuestionIndex` embeds indexed question labels (enriched with form-name context) with a HuggingFace embedding model (`BAAI/bge-small-en-v1.5` by default) for `semantic_search`, with form-weight normalization so a handful of over-represented forms can't flood every result set.

`retrieval/examples.py`'s `SeedExampleIndex` is a two-stage retrieve-then-rerank index over the seed corpus: cosine similarity first, then an optional `CrossEncoder` reranker (`BAAI/bge-reranker-base`, env-gated — falls back to an augmented-embedding heuristic if not configured), then NL-prefix deduplication.

3.6 Knowledge

`knowledge/semantics_registry.py` is the declared single source of truth for domain facts that would otherwise be duplicated across prompts, rules, and the validator — status lifecycles, join-cast directions, counting contracts, financial routing. This exists specifically because an earlier real incident (a duplicated "overdue" definition drifting out of sync in five places) went undetected for a period. `verify_consistency()`, run automatically at server startup, now scans prompt/rule files for hardcoded values that have drifted from the registry's canonical definitions and logs a warning — a regression guard against that exact failure mode recurring silently.

3.7 Security

`security/auth.py` handles authentication only (bearer-token bookkeeping via an in-memory token map). Authorization is separate: `security/rbac.py`'s `build_scope_filter()` turns a user's permission list into a SQL WHERE fragment — unrestricted if a "view all" permission is present, role-scoped (inspector/inspectee, unioned for dual-role users) otherwise, or an explicit `'BLOCK'` if no matching permission exists. The scope filter is enforced **twice**: once before any LLM call in `AgentOrchestrator.query()` (and embedded in the system prompt as a "never override" constraint), and again after generation via `validator.enforce_scope()` as a defense-in-depth backstop against the LLM ignoring the prompt-level instruction.

3.8 Chart Advisor NO LLM

`charts/advisor/` recommends a chart type purely from the shape of the result set — column types (date/numeric/category detection), row/column cardinality, sign mixing, and similar heuristics (e.g. a single-row numeric result becomes a KPI/gauge, two categorical dimensions become a heatmap, day-level dates become a calendar heatmap). No model call is involved anywhere in this package, confirming the README's claim.

3.9 API Layer

`api/app.py`'s lifespan warms both domain orchestrators and the domain router at startup (so the first real request isn't paying model/embedding load time), then runs the consistency check from §3.6. The README documents four endpoints; the actual route set is larger:

Endpoint	Purpose	In README?
`POST /api/chat`	Main entry point — message in, answer/SQL/steps/chart out.	Yes
`POST /api/chart`	Force-regenerate a chart with column overrides.	Yes
`GET /api/columns/{session_id}`	Available columns + semantic types for the last query.	Yes
`POST /api/page`	Server-side pagination for large result sets.	Yes
`GET /api/sql/{session_id}`	Fetch the SQL behind the last result.	NO
`GET /api/health, /api/ready`	Startup/liveness probes.	NO
`GET /api/validator-stats`	Validator hit-rate telemetry.	NO
`POST /api/login, /api/logout`	Auth.	NO
`GET /api/me/permissions`	Current user's RBAC permission list.	NO
`POST /api/transcribe, /api/tts`	Voice input / spoken-answer playback.	NO

`api/deps.py` centralizes an in-memory session store (`_sessions`, keyed by session+user) holding the `ConversationSession` plus cached last-query rows/columns/SQL for pagination and chart requests — this cache is why `/api/page` and `/api/chart` don't need to re-run the original query.

3.10 Session / Conversation State

`agent/orchestrator/session.py`'s `ConversationSession` holds a capped turn history (8 turns), a persistent `current_scope` pointer that deliberately survives turn eviction (so "how much history is shown" and "what's currently focused" are decoupled), and an append-only entity-memory list (capped at 30) for recalling named entities from further back than the turn window. Domain-exclusive scope types are gated so scope from an inspection conversation can never leak into an audit query or vice versa.

3.11 LLM Provider Abstraction

`llm/base.py` defines a minimal client protocol — anything with a `.complete(prompt)` method. `llm/factory.py` is *role*-based rather than provider-based: `get_reasoning_llm`/`get_sql_llm` build Ollama clients with role-specific tuning (context window, temperature 0.0, stop tokens); `get_reasoning_llm_anthropic`/`get_sql_llm_anthropic` are the Claude equivalents. A single settings flag decides which factory pair gets used — no other code in the system needs to know which provider is active, since every call site only ever calls `.complete()`.

4 Frontend Architecture

4.1 App Shell & Navigation

Angular 21.2, entirely standalone components — no `NgModules` remain in the codebase. A single `AppComponent` owns login, a startup splash screen, theme toggling, and tab-switching between four views (`chat`, `dashboard`, `saved`, `schema`) via a plain component-local string field and `*ngSwitchCase` — not the Angular Router.

> **DEAD CODE FOUND** `@angular/router` is a dependency and `guards/permission.guard.ts` implements a working `CanActivateFn`, but there is no `app.routes.ts` and `provideRouter` is never called anywhere in the app. The router and its guard are unused — navigation is entirely the manual tab-switch described above.

4.2 Chat Experience

`components/chat-window/` (912 lines) is the primary surface: message list, input with slash-command autocomplete, voice input (recorded audio → `/api/transcribe`), and text-to-speech playback of answers (`/api/tts`). It talks to the backend exclusively through `ChatService`. There is **no streaming** — each question is a single blocking HTTP POST to `/api/chat`; a rotating "thinking" label ("Parsing your question…" → "Generating SQL…") is purely client-side UI flavor timed against the wait, not driven by real backend progress events.

4.3 Data Display Components

Component	Role
`result-table`	Renders rows/columns with server-side pagination (`POST /api/page`). Detects 2-column entity/count summaries and turns row clicks into follow-up questions ("drilldown").
`column-picker`	Modal for choosing X/Y axis columns before drawing a custom chart on a wide result set.
`chart-display`	Wraps Apache ECharts (963 lines) — quick-render mode and a full control mode with chart-type switching, axis controls, fullscreen, and conflict banners when a requested chart type differs from the advisor's recommendation.
`dashboard-display`	Per-message composite of KPI cards + chart + table, shown when the backend response flags `dashboard.show`.

Feed chain: a result table → user triggers `/chart` or a drilldown click → column picker (only if more than 4 columns) → chart-display renders the returned config. `dashboard-display` is a separate composite path, not downstream of the chart-display flow.

4.4 Supporting UI

Component	Role
`history-sidebar`	Conversation list grouped by Today/Yesterday/Earlier — client-side only, no backend persistence (lost on reload).
`debug-panel`	Reasoning steps, generated SQL with syntax highlighting, token/timing metrics — the visible half of the backend's `steps` response field.
`slash-autocomplete`	Dropdown suggestions as the user types a `/` command.
`dashboard-view` `MOCK DATA`	The top-nav "Dashboard" tab — hardcoded constants, not wired to any backend endpoint.
`saved-view` `MOCK DATA`	The "Saved Queries" tab — hardcoded constants, not backend-connected.
`schema-view` `MOCK DATA`	The "Schema" tab — a browser over a hardcoded schema constant, not fetched live.

> **THREE OF THE FOUR TOP-LEVEL TABS ARE DEMO CONTENT** Only the Chat tab is fully backend-driven. Dashboard, Saved Queries, and Schema all render static, hardcoded data — worth knowing before assuming any of the three reflects live system state.

4.5 Services, Models, Cross-Cutting Concerns

State management is RxJS `BehaviorSubject`s inside three `@Injectable({providedIn:'root'})` services — no signals, no NgRx or similar state library anywhere in the codebase. `ChatService` owns chat/pagination/session-clear calls; `AuthService` wraps `sessionStorage` (deliberately per-tab, not `localStorage`, so multiple tabs can hold independent sessions) and the permissions endpoint; `SlashCommandService` parses commands and owns chart/SQL-fetch/CSV-export. `models/chat.model.ts` defines the shared response shapes (`ChatResponse`, `ResultData`, `ChartConfig`, `DashboardCard`, `Message`). An HTTP interceptor attaches the bearer token to every request except login and force-reloads on a 401; a structural directive (`*appHasPermission`) conditionally shows template content by RBAC permission.

4.6 Build & Tooling

Angular 21.2, TypeScript 5.9, RxJS 7.8, Vitest for testing. Charting is Apache ECharts 6.1 used directly (no wrapper library), confirming the backend README's claim. No UI component library — all styling is hand-rolled inline templates/SCSS. `proxy.conf.json` proxies `/api` to `http://localhost:8000` during development, matching the root README; the frontend's own README states port 8001, which is the stale figure between the two documents.

5 Frontend ↔ Backend Contract

Every endpoint the frontend actually calls, cross-checked against both READMEs:

Endpoint	Called from	Purpose
`POST /api/chat`	ChatService	Ask a question, get answer + SQL + rows + chart + steps
`POST /api/page`	ChatService, result-table	Fetch the next page of a cached result set
`POST /api/chart`	SlashCommandService, chart-display	Regenerate a chart with column overrides
`GET /api/columns/{id}`	SlashCommandService, chart-display	Column metadata for the picker/chart controls
`GET /api/sql/{id}`	SlashCommandService	Fetch SQL behind the last result (`/sql` command)
`DELETE /api/sessions/{id}`	ChatService	Clear a conversation (`/clear` command)
`POST /api/login, /api/logout`	AppComponent, AuthService	Session auth
`GET /api/me/permissions`	AuthService	RBAC permission list for the current user
`GET /api/health`	AppComponent	Startup splash-screen readiness check
`POST /api/transcribe, /api/tts`	chat-window	Voice input / spoken answer playback

6 Summary of Notable Findings

- **Model names in the README are stale.** The live system runs qwen3.5:9b (reasoning) / qwen2.5-coder (SQL generation), not Llama 3.1 8B / DeepSeek Coder V2, and also supports a Claude-only mode.
- **The Angular Router is installed but unused.** Navigation is a manual tab switch; the router guard that exists is dead code.
- **Three of four top-level frontend tabs are static demo data** — Dashboard, Saved Queries, and Schema are not backend-connected. Only Chat is live.
- **The API surface is larger than documented** — 9 endpoints exist beyond the 4 the backend README lists (health/ready checks, auth, voice, SQL/session lookups).
- **Conversation history has no server-side persistence** — both the backend's session store and the frontend's sidebar are in-memory only.

None of these are defects — several are deliberate scope boundaries (demo tabs not yet built out, router not yet needed for a single-page tabbed app) — but they're worth knowing explicitly rather than inferring from the documentation alone.